

Week 6: Sync & Concurrency, Cont.

TABLE OF CONTENTS

Complete Table of Contents

- 1 [Weekly Summary and Where are we?](#)
 - a [Topics](#)
 - b [Assignments](#)
 - c [Reading Summary](#)
 - d [Announcements](#)
- 2 [Synchronization, continued](#)
 - a [Review from last week](#)
- 3 [2. Deadlock](#)
 - a [Example: Simple Deadlock](#)
 - b [Example: A more subtle example](#)
 - c [When does deadlock happen?](#)
 - d [What can we do about deadlock?](#)
 - a [Option 1: Ignore it!](#)
 - b [Option 2: Detect and Recover](#)
 - c [Option 3: Avoid it Algorithmically](#)
 - d [Option 4: Avoid Necessary Pre-conditions by being careful with our Coding](#)
 - e [Option 5: Use Static and Dynamic Detection Tools](#)
- 4 [3. Other concurrency issues](#)
 - a [Making Progress](#)
 - a [Livelock](#)
 - b [Starvation](#)
 - c [Priority Inversion](#)
 - b [Other Concurrency bugs](#)
 - a [Atomicity-violation bugs](#)
 - b [Order-violation bugs](#)
 - c [Performance vs. Complexity](#)
- 5 [Other synchronization mechanisms](#)
 - a [Semaphores](#)
 - b [Barrier](#)
 - c [Spinlocks](#)
 - a [Example: Spinlock, Broken Implementation](#)
 - b [Example: Correct Spin-Lock Implementation](#)
 - d [Reader-Writer locks](#)
 - e [Read-copy-update \(RCU\)](#)
 - f [Deterministic Multithreading](#)
- 6 [Summary/Wrap-Up](#)
 - a [Acknowledgements](#)

Agenda

Weekly Summary and Where are we?

Topics

- **Last Week:** More about Synchronization
- **This Week:** Finish up Synchronization,
- **Next Week:** Introduce Virtual Memory

Assignments

- **Lab 3** is out; due 13 Feb 2026– THIS Friday.
- Reminder: Regrade requests can be requested until the Friday after the grades are released.

Reading Summary

- OSTEP Chapter 25: Dialogue
- OSTEP Chapter 26: Concurrency and Threads
- OSTEP Chapter 27: Thread API
- OSTEP Chapter 28: Locks

Announcements

- Please be checking Piazza!
- How's Lab 3 going?

Synchronization, continued

Review from last week

List 5 things!

- 1) Conditional Variables — wait for a condition to be true.
- 2) Critical section — code that different threads want to use shared resource
- 3) Locks/Mutex: Want to use shared resource
- 4) Monitors — Combo of locks and cond. var.
- 5) Monitors — Combo of locks and cond. var.

Summary poem from last week:

In code's dance, we wait with glee,
Conditional variables set us free.
Locks guard treasures, no race to fight,
Critical sections sleep tight at night.
Monitors blend the best of both,
Syncing threads with joyful oath.
Concurrency's magic, a happy quirk,
Together they work—oh, what a perk! 🥳🎁

What part of the poem shows how we keep threads from stepping on each other's toes?

- condition variable: `cond_wait(Cond, Mutex)`
- always use "while" instead of "if"
- has to be a single interface (vs. release/wait/acquire)
- Signal vs. Broadcast
- both signal and broadcast wake up threads.
 - `signal()` wakes up one thread
 - `broadcast()` wakes up all threads

QUESTION

Can we replace `SIGNAL` with `BROADCAST`, given our monitor semantics?

Answer

YES.

Example:

Memory Allocator

- threads: `malloc(5), free(5)`
`malloc(16), free(16)`

- 2 threads: waiting.

- Third thread: `free(10,000)`.

QUESTION

Can we replace `BROADCAST` with `SIGNAL`?

Answer

Not Always.

- Monitor
 - an object (= mutex + condition variables)
 - should be a language primitive
 - C: a programming convention
- "a safe way": Monitor + 6 rules + 4-step design approach
 - 6 rules:
 - must memorize!
 - 4-step design approach
- Dangerous concurrency world
 - never ever try to create your own synchronization; they are likely to be broke(why? do you understand your hardware? don't build a castle on a swamp)
- Don't manipulate synchronization variables or shared state variables in the code associated with a thread; do it with the code associated with a shared object.
 - Threads tend to have "main" loops. These loops tend to access shared objects. *However*, the "thread" piece of it should not include locks or condition variables. Instead, locks and CVs should be encapsulated in the shared objects.
- Why?
 - (a) Locks are for synchronizing across multiple threads. Doesn't make sense for one thread to "own" a lock.
 - (b) Encapsulation – details of synchronization are internal details of a shared object. Caller should not know about these details. "Let the shared objects do the work."
 - Common confusion: trying to acquire and release locks inside the threads' code (i.e., not following this advice). Bad idea! Synchronization should happen within the shared objects. Mantra: "let the shared objects do the work".
 - Note: our first example of condition variables – [Week 5 Notes, Example Producer/Consumer with Mutexes and Condition Variables](#) – doesn't actually follow the advice, but that is in part so you can see all of the parts working together.
- Different way to state what's above:
 - You want to decompose your problem into objects, as in object-oriented style of programming.
 - Thus:
 - (1) Shared object encapsulates code, synchronization variables, and state variables
 - (2) Shared objects are separate from threads

2. Deadlock

See code examples below.

EXAMPLE: SIMPLE DEADLOCK

Simple Deadlock Example

```

T1:
  acquire(mutexA);
  acquire(mutexB);

  // do some stuff

  release(mutexB);
  release(mutexA);

T2:
  acquire(mutexB);
  acquire(mutexA);

  // do some stuff
  
```

```
release(mutexA);
release(mutexB);
```

EXAMPLE: A MORE SUBTLE EXAMPLE

More Subtle Example

- Let `M` be a monitor (shared object with methods protected by mutex)
- Let `N` be another monitor

```
class M {
private:
    Mutex mutex_m;
```

← Uses Memory Allocator

```
// instance of monitor N
N another_monitor;
```

```
// Assumption: no other objects in the system hold a pointer
// to our "another_monitor"
```

```
public:
    M();
    ~M();
    void methodA();
    void methodB();
```

```
};
```

```
class N {
private:
    Mutex mutex_n;
    Cond cond_n;
    int navailable;
```

```
public:
    N();
    ~N();
    void* alloc(int nwanted);
    void free(void*);
```

```
}
```

```
int
```

```
N::alloc(int nwanted) {
    acquire(&mutex_n);
    while (navailable < nwanted) {
        wait(&cond_n, &mutex_n);
    }
```

← Memory Allocator

```
// peel off the memory
navailable -= nwanted;
release(&mutex_n);
```

```
}
```

```
void
```

```
N::free(void* returning_mem) {
    acquire(&mutex_n);
```

```
// put the memory back
```




```

    navailable += returning_mem;

    broadcast(&cond_n, &mutex_n);

    release(&mutex_n);
}

void
M::methodA() {
    acquire(&mutex_n);

    void* new_mem = another_monitor.alloc(int nbytes);

    // do a bunch of stuff using this nice
    // chunk of memory n allocated for us

    release(&mutex_n);
}

void
M::methodB() {
    acquire(&mutex_n);

    // do a bunch of stuff
    another_monitor.free(some_pointer);

    release(&mutex_n);
}

```

What if we
remove the mutex?
Does that cause
a problem?

QUESTION

What's the problem here?

Answer

- lesson: dangerous to hold locks (M's mutex that we see in the code above) when crossing abstraction barriers
- deadlocks without mutexes:
 - real issue is resources and how/when they are required/acquired
 - (a) [draw bridge example]
 - bridge only allows traffic in one direction
 - Each section of a bridge can be viewed as a resource.
 - If a deadlock occurs, it can be resolved if one car backs up (preempt resources and rollback).
 - Several cars may have to be backed up if a deadlock occurs.
 - Starvation is possible.
 - (b) another example:
 - one thread/process grabs disk and then tries to grab scanner
 - another thread/process grabs scanner and then tries to grab disk

When does deadlock happen?

All of the following four conditions must happen for deadlock to happen:

- 1 **Mutual exclusion/bounded resources:** There are finite number of threads that can simultaneously use a resource (e.g. keyboard)
- 2 **Hold-and-wait:** A thread holds one resource while waiting for another. (aka "multiple independent requests")
- 3 **No preemption:** Once a thread acquires a resource, it holds it until it releases it (it can't be revoked)

- 4 **Circular wait:** There is a set of waiting threads such that each thread is waiting for a resource held by another.

What can we do about deadlock?

A few options:

- Ignore it
- Detect it, then recover
- Use an algorithm to be smart about acquiring/releasing
- Avoid necessary preconditions with our coding
- Use static and dynamic detection tools/analysis

Valgrind

Option 1: Ignore it!

Wait until it happens, see what can be done. ٩(ツ)٩

Option 2: Detect and Recover

Sometimes it's difficult or expensive to enforce enough structure on a system to prevent all deadlocks.

In order to do this, we need:

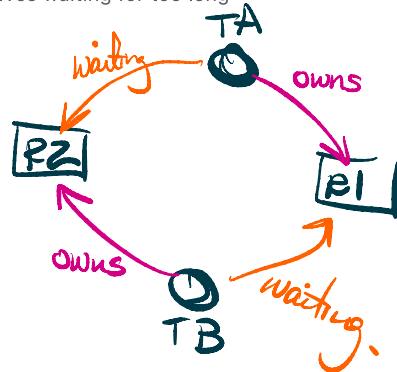
- A way to detect deadlock
- A way to recover from deadlock, without too much harm

Ways we might recover:

- Could just kill the process, but this isn't always as harmless as it seems
 - Imagine if a thread in the process is holding a lock on a kernel process
- Proceed without the resource
 - Consider the web: If it takes too long for a site to respond, users leave.
 - Example: Buying something at Amazon
 - Normal operation: the software checks inventory to ensure stock before completing a sale.
 - Assume that the check inventory step fails or deadlocks:
 - Give up on waiting for the inventory
 - Go ahead and complete the order
 - Queue a background task to check if the item is actually available
 - If the item was not available, send an apology to the customer
- Transactions: Rollback and retry
 - We'll talk more about transaction in the future
 - Choose a thread to rollback:
 - Stop the thread
 - Undo whatever the thread has already done
 - Let other threads proceed
 - After some other threads have started moving, restart the rolled-back thread

Ways we might detect:

- We could keep it simple, and just assume deadlock if we find ourselves waiting for too long
 - Might get false positives, but that might be okay
- We could keep track of a resource graph
 - Imagine a graph with each thread and each resource is a node
 - Edges:
 - a thread owns a resource
 - a thread is waiting for a resource
 - Any cycles indicate deadlock → move to recovery



Option 3: Avoid it Algorithmically

Use the Banker's Algorithm to eliminate wait-while-holding.

- Essentially, wait until all needed resources are available and acquire them atomically at the beginning of an operation.
 - In contrast to acquiring resources as we continue
- Con: We need to know all the resources we might need by each process
- It's complex, but simplified versions are commonly used

Why not just use a queue in front of all shared resources?
- instead of using locks, w/ the danger of deadlock.

- It looks like this:

```
ResourceMgr::Request(ResourceID resc,
                    RequestorID thrd) {
    acquire(&mutex);
    assert(system in a safe state);
    while (state that would result from giving
           resource to thread is not safe) {
        wait(&cv, &mutex);
    }
    update state by giving resource to thread
    assert(system in a safe state);
    release(&mutex);
}
```

What does it mean for the system to be "in a safe state"?

- For any possible sequence of resource requests, there is at least one safe sequence of processing the requests that eventually succeeds in grant all pending and future requests.
- an example:
 - a bank has 10 coins
 - three customers (max coins needed): A (9), B (4), and C (7)
 - current status:

	has	max
A	2	9
B	2	4
C	7	7

bank: 3 1 5 7 1

SAFE: Yes, because there is a way to give all the resources.

QUESTION

Is current state safe?

Answer

QUESTION

If A requests and gets another coin, is the state then safe?

Answer

See details at [Wiki](#).

- disadvantage to banker's algorithm:
 - requires every single resource request to go through a single broker
 - requires every thread to state its maximum resource needs up front.
 - unfortunately, if threads are conservative and claim they need huge quantities of resources, the algorithm will reduce concurrency

Option 4: Avoid Necessary Pre-conditions by being careful with our Coding

We can try to avoid the 4 conditions by coding more carefully.

- Negating #1 (kinda), mutual exclusion: *bounded resources*
- put a queue in front of resources, like the printer
- Negating #2 (kinda), hold-and-wait:
 - Release the lock when calling out of the module

→ for deadlock.

Red flag!

- either cancel "hold": abort when conflicting
- or cancel "wait": hold *all* resources at once
- Negating #3 (kinda), no pre-emption:
 - Choose to pre-empt resources (forcibly reclaim resources held by a different thread)
 - Ex: consider physical memory: virtualized with VM, can take physical page away and give to another process!
- Negating #4, circular wait:
 - In practice, this is what people do
 - Big idea: partial order on locks
 - Establishing an order on all locks and making sure that every thread acquires its locks in that order
 - Requires developers to follow a convention
 - Why this works:
 - can view deadlock as a cycle in the resource acquisition graph
 - partial order implies no cycles and hence no deadlock
 - Drawbacks
 - 1 Difficult to represent conditional variables inside this framework. * works best only for locks.
 - 2 Picking and obeying the order on *all* locks requires that modules make public their locking behavior, and requires them to know about other modules' locking. This can be painful and error-prone.
 - Example: see Linux's `filemap.c` example below (source is [here](#)); this is complexity that arises by the need for a locking order

• give every resource an order.
• take resources in that order.

```

/*
 * linux/mm/filemap.c
 *
 * Copyright (C) 1994-1999 Linus Torvalds
 */

/*
 * This file handles the generic file mmap semantics used by
 * most "normal" filesystems (but you don't /have/ to use this:
 * the NFS filesystem used to do this differently, for example)
 */

// (AHS: #includes and other notes omitted for brevity)

/*
 * Lock ordering:
 *
 * ->i_mmap_rwsem (truncate_pagecache)
 *   ->private_lock (__free_pte->__set_page_dirty_buffers)
 *     ->swap_lock (exclusive_swap_page, others)
 *       ->i_pages lock
 *
 * ->i_mutex
 *   ->i_mmap_rwsem (truncate->unmap_mapping_range)
 *
 * ->mmap_sem
 *   ->i_mmap_rwsem
 *     ->page_table_lock or pte_lock (various, mainly in memory.c)
 *       ->i_pages lock (arch-dependent flush_dcache_mmap_lock)
 *
 * ->mmap_sem
 *   ->lock_page (access_process_vm)
 *
 * ->i_mutex (generic_perform_write)
 *   ->mmap_sem (fault_in_pages_readable->do_page_fault)
 *
 * bdi->wb.list_lock
 * sb_lock (fs/fs-writeback.c)
 * ->i_pages lock (__sync_single_inode)

```

Topo-Sort.
DAG.



shirt → coat
pants → belt
socks → shoes

```

*
* ->i_mmap_rwsem
*   ->anon_vma.lock (vma_adjust)
*
* ->anon_vma.lock
*   ->page_table_lock or pte_lock (anon_vma_prepare and various)
*
* ->page_table_lock or pte_lock
*   ->swap_lock (try_to_unmap_one)
*   ->private_lock (try_to_unmap_one)
*   ->i_pages lock (try_to_unmap_one)
*   ->zone_lru_lock(zone) (follow_page->mark_page_accessed)
*   ->zone_lru_lock(zone) (check_pte_range->isolate_lru_page)
*   ->private_lock (page_remove_rmap->set_page_dirty)
*   ->i_pages lock (page_remove_rmap->set_page_dirty)
*   bdi.wb->list_lock (page_remove_rmap->set_page_dirty)
*   ->inode->i_lock (page_remove_rmap->set_page_dirty)
*   ->memcg->move_lock (page_remove_rmap->lock_page_memcg)
*   bdi.wb->list_lock (zap_pte_range->set_page_dirty)
*   ->inode->i_lock (zap_pte_range->set_page_dirty)
*   ->private_lock (zap_pte_range->__set_page_dirty_buffers)
*
* ->i_mmap_rwsem
*   ->tasklist_lock (memory_failure, collect_procs_ao)
*/

static int page_cache_tree_insert(struct address_space *mapping,
struct page *page, void **shadowp)
{
    struct radix_tree_node *node;
    ...

```

It's a bit ironic that the "best" approach for preventing deadlock is one that goes against so many best programming practices...

Option 5: Use Static and Dynamic Detection Tools

- (if you're interested) See, for example, these citations, citations therein, and papers that cite them:

Engler, D. and K. Ashcraft. RacerX: effective, static detection of race conditions and deadlocks. Proc. ACM Symposium on Operating Systems Principles (SOSP), October, 2003, pp237-252. <http://portal.acm.org/citation.cfm?id=945468>

Savage, S., M. Burrows, G. Nelson, P. Sobalvarro, and T. Anderson. Eraser: a dynamic data race detector for multithreaded programs. ACM Transactions on Computer Systems (TOCS), Volume 15, No 4., Nov., 1997, pp391-411. <http://portal.acm.org/citation.cfm?id=265927>

- Active (and long!) area of systems research
- Disadvantage to dynamic checking: slows program down
- Disadvantage to static checking: many false alarms (tools says "there is deadlock", but in fact there is none) or else missed problems
- Note that these tools get better every year. I believe that Valgrind has a race and deadlock detection tool

3. Other concurrency issues

Making Progress

Deadlock was one kind of progress (or liveness) issue. Here are others...

Livelock

- Livelock
 - two threads waiting on each other: deadlock
 - each backs off when finding another process
 - a possibility: all processes keep aborting and make no progress

[draw a curly bridge]

Starvation

- Starvation
 - thread waiting indefinitely (if low priority and/or if resource is contended)
 - for example, merging to a busy road with a yield sign

Priority Inversion

- Priority inversion
 - T1, T2, T3: (highest, middle, lowest priority)
 - T1 wants to get lock, T2 runnable, T3 runnable and holding lock
 - System will preempt T3 and run highest-priority runnable thread, namely T2
 - Solutions:
 - Temporarily bump T3 to highest priority of any thread that is ever waiting on the lock
 - Disable interrupts, so no preemption (T3 finishes) ... not great because OS sometimes needs control (not for scheduling, under this assumption, but for handling memory [page faults], etc.)
 - Don't handle it; structure app so only adjacent priority processes/threads share locks
 - Happened in real life, see: <https://www.microsoft.com/en-us/research/people/mbj/just-for-fun/> (search for Pathfinder)
 - Mars pathfinder experience total system resets
 - what happened:
 - data gathering thread (low priority)
 - communication thread (medium priority)
 - management thread (high priority)
 - data gathering and management threads may acquire the same mutex
 - classic priority inversion

Other Concurrency bugs

(b) other concurrency bugs

Atomicity-violation bugs

```
T1:
  if (thd->info) {
    ...
    use(thd->info);
    ...
  }

T2:
  thd->info = NULL
```

Order-violation bugs

```
T1:
  void init() {
```

```

    thd = createThread(...)
}

T2:
void main() {
    state = thd->info
}

```

["Learning from Mistakes – A Comprehensive Study on Real World Concurrency Bug Characteristics"](#)

Performance vs. Complexity

- Coarse locks limit available parallelism

[(still, you should design this way at first!!!)]

the fundamental issue with coarse-grained locking is that only one CPU can execute anywhere in the part of your code protected by a lock. If your critical section code is called a lot, this may reduce the performance of an expensive multiprocessor to that of a single CPU.

if this happens inside the kernel, it means that applications will inherit the performance problems from the kernel

- ... but fine-grained locking leads to complexity and hence bugs (like deadlock)
 - although finer-grained locking can often lead to better performance, it also leads to increased complexity and hence risk of bugs (including deadlock).
 - e.g. the ordering of locks in the linux code example excerpted above

Other synchronization mechanisms

Semaphores

- mentioned last time
- first general-purpose synchronization primitive
- (you should not use it)

Barrier

- an example: GPU * many cores, literally thousands * no cache coherence (unlike CPU) * after barrier, all threads are synced

Spinlocks

- when acquiring the lock, the thread "spins".
- implementation (see code blocks below)
- atomic instructions are expensive * some detailed experiments here: <https://arxiv.org/pdf/2010.09852.pdf>

EXAMPLE: SPINLOCK, BROKEN IMPLEMENTATION

```

struct Spinlock {
    int locked;
}

void acquire(Spinlock *lock) {

```



```

while (1) {
    if (lock->locked == 0) { // A
        lock->locked = 1; // B
        break;
    }
}

void release (Spinlock *lock) {
    lock->locked = 0;
}

```

What's the problem?

- Two `acquire()`s on the same lock on different CPUs might both execute line A, and then both execute B. Then both will think they have acquired the lock. Both will proceed. That doesn't provide mutual exclusion.

EXAMPLE: CORRECT SPIN-LOCK IMPLEMENTATION

Relies on atomic hardware instruction. For example, on the x86-64, doing `xchg addr, %rax` does the following:

- (i) freeze all CPUs' memory activity for address `addr`
- (ii) `temp <-- *addr`
- (iii) `*addr <-- %rax`
- (iv) `%rax <-- temp`
- (v) un-freeze memory activity

```

/* pseudocode */
int xchg_val(addr, value) {
    %rax = value;
    xchg (*addr), %rax
}

/* bare-bones version of acquire */
void acquire (Spinlock *lock) {
    pushcli(); /* what does this do? */
    while (1) {
        if (xchg_val(&lock->locked, 1) == 0)
            break;
    }
}

void release(Spinlock *lock){
    xchg_val(&lock->locked, 0);
    popcli(); /* what does this do? */
}

/* optimization in acquire; call xchg_val() less frequently */
void acquire(Spinlock* lock) {
    pushcli();
    while (xchg_val(&lock->locked, 1) == 1) {
        while (lock->locked) ;
    }
}

```

The above is called a *spinlock* because `acquire()` spins. The bare-bones version is called a "test-and-set (TAS) spinlock"; the other is called a "test-and-test-and-set spinlock".

The spinlock above is great for some things, not so great for others. The main problem is that it *busy waits*: it spins, chewing up CPU cycles. Sometimes this is what we want (e.g., if the cost of going to sleep is greater than the cost of spinning for a few cycles waiting for another thread or process to relinquish the spinlock). But sometimes this is not at all what we want (e.g., if the lock would be held for a while: in those cases, the CPU waiting for the lock would waste cycles spinning instead of running some other thread or process).

NOTE: the spinlocks presented here can introduce performance issues when there is a lot of contention. (This happens even if the programmer is using spinlocks correctly.) The performance issues result from cross-talk among CPUs (which undermines caching and generates traffic on the memory bus). If we have time later, we will study a remediation of this issue (search the Web for "MCS locks").

ANOTHER NOTE: In everyday application-level programming, spinlocks will not be something you use (use mutexes instead). But you should know what these are for technical literacy, and to see where the mutual exclusion is truly enforced on modern hardware.

Reader-Writer locks

- recall the "database problem" in Week 5
- usage (*pseudocode*):

```
rwlock_t lock
acquire_rdlock(&lock)
acquire_wrlock(&lock)
unlock(&lock)
```

Read-copy-update (RCU)

- do we have to block readers when modifying a data structure?
- answer: No, using RCU
- an example of deletion in double-linked list [show slides] TODO(ahs)
- basic idea:
 - readers work as normal (as if there is no concurrent updaters)
 - updaters
 - make copies
 - maintain both old and new version for a "grace period"
 - delete the old version
- see also: <https://cs.nyu.edu/~mwaldfish/classes/14fa/ref/rcu2.pdf>

Deterministic Multithreading

- concurrent programs have "infinite" possible schedules (hence we don't know what may go wrong)
- BUT, we do know which schedules are okay (for example, by testing)
- idea: avoid unseen interleavings and schedules which are potentially buggy
- see also: "Stable Deterministic Multithreading through Schedule Memoization" <https://www.cs.columbia.edu/~junfeng/11sp-w4118/lectures/tern.pdf>
- A useful book for concurrency ninjas: Paul E. McKenney, "Is Parallel Programming Hard, And, If So, What Can You Do About It?" <https://mirrors.edge.kernel.org/pub/linux/kernel/people/paulmck/perfbook/perfbook-e2.pdf>

Summary/Wrap-Up

- 1) **Deadlock!**
- 2) **Broadcast/Signal**
- 3) **Banker's Algorithm**
- 4) **Partial-ordering**
- 5) **4 req. for deadlock - circular waiting.**

Let's put these in Khanmigo and get a summary.